

Third Module

Marco Barberio

Master Degree in Computer Science

`m.barberio2@studenti.unipi.it`

Parallel and Distributed Systems: Paradigms and Models, Academic Year: 2025/2026

1 OpenMP parallelization strategy

The OpenMP implementations preserve the same partitioned hash join structure used in Module 2. Therefore, this module does not change the algorithmic pipeline, but focuses on how the available parallelism can be expressed using different OpenMP constructs.

The first implementation is based on loop-level parallelism. The histogram and scatter phases are parallelized using `#pragma omp parallel for` with static scheduling, since the input relation is split into disjoint chunks of similar size and the amount of work per chunk is expected to be relatively regular. Static scheduling also keeps the overhead low, because the assignment of chunks to threads is decided in advance.

In the local join phase, instead, the loop over partitions uses dynamic scheduling. This choice was adopted because different partitions may have different processing costs, especially under skewed workloads. Even if the number of partitions is fixed, the join cost can depend on the number of records in each partition, the amount of duplicate keys, and the number of successful matches. With dynamic scheduling, threads receive new partitions as soon as they complete their current work, reducing the risk that some threads remain idle while others are still processing expensive partitions. The second implementation is based on OpenMP tasks. In this version, the histogram and scatter phases create tasks over chunks of the input relation, while the local join phase creates one task for each partition.

In both implementations, the final accumulation of the per-partition results is kept sequential, since it represents a small fraction of the total execution time and avoids introducing unnecessary synchronization. The final output consists of join count, checksum1, and checksum2

1.1 Skewed workload generation

In addition to the uniform workload, I introduced a skewed workload to evaluate the behavior of the OpenMP implementations under load imbalance. The uniform workload generates keys pseudo-randomly in the range $[0, \text{max_key})$.

For the skewed workload, a small subset of partitions is selected as hot partitions. Before generating the records, the generator separates the keys into two sets: keys that map to hot partitions and keys that map to cold partitions. Then, for each generated record, a configurable percentage of keys is sampled from the hot set, while the remaining records are sampled from the cold set.

In this way, a large fraction of the input records is forced to map to a small number of partitions, producing an intentionally imbalanced workload.

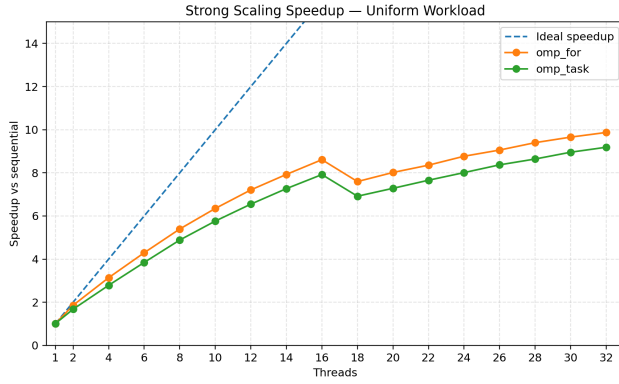
2 Experimental Results

All experiments were conducted on node-08 of the spmcluster. The number of partitions was fixed to $P = 1024$, and keys were generated in the range $[0, 100000)$. For each configuration, execution times were averaged over ten seeds, from 0 to 9. Both OpenMP implementations were tested on uniform and skewed workloads.

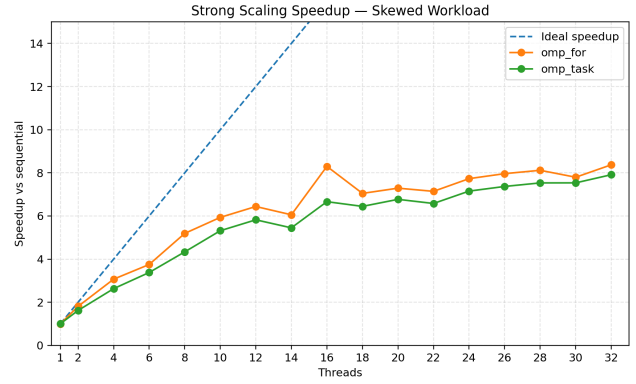
For strong scaling, the input size was fixed to $N_R = N_S = 10^7$, while the number of threads was varied from 1 to 32. For weak scaling, the input size was increased proportionally to the number of threads, starting from $N_R = N_S = 20000$ with one thread and reaching $N_R = N_S = 640000$ with 32 threads. For skewed workloads, 16 hot partitions were used, with 90% of the records assigned to keys mapping to those partitions. In the skew-sensitivity experiment, the skew percentage was varied in $\{0, 50, 75, 90, 95, 99\}$, using $N_R = N_S = 10^7$ and $T = 8$ threads.

2.1 Strong Scaling

Figure 1 reports the speedup with respect to the sequential baseline, showing the uniform and skewed workloads separately. Overall, both OpenMP implementations improve over the sequential execution, but remain below the ideal linear speedup, as already observed in Module 2 with the C++ threads implementation.



(a) Uniform workload



(b) Skewed workload

Figure 1: Strong scaling speedup of the OpenMP loop-based and task-based implementations under uniform and skewed workloads.

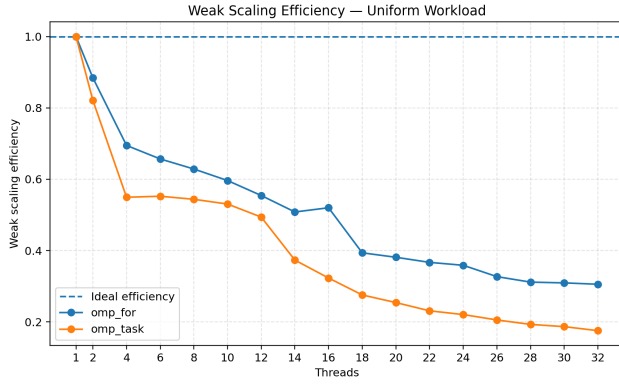
On the uniform workload, the loop-based implementation achieves the best scalability, reaching almost $10\times$ speedup at 32 threads. This result is very close to the one obtained in Module 2, where the explicit C++ threads version also reached approximately $10\times$ speedup at 32 threads. This suggests that, for a regular workload, OpenMP loop-level parallelism is able to match the behavior of the manually managed thread-based implementation.

On the skewed workload, both implementations still scale, but the maximum speedup is lower. This is expected because the local join cost becomes concentrated on fewer partitions. Nevertheless, the loop-based implementation remains the most robust one, mainly thanks to the dynamic scheduling used in the partition-wise join phase. The task-based version is slightly less effective, because its additional scheduling overhead is not compensated by a sufficient load-balancing improvement in these experiments.

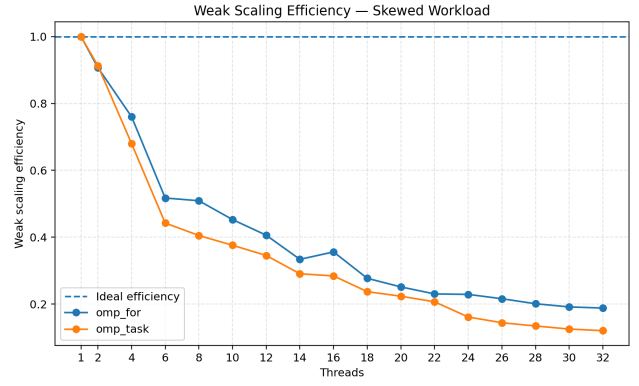
2.2 Weak Scaling

Figure 2 shows the weak scaling efficiency for the uniform and skewed workloads separately. In both cases, efficiency decreases as the number of threads increases, following the same general trend already observed in Module 2.

On the uniform workload, the loop-based implementation maintains the highest efficiency across all thread counts. The task-based implementation is less efficient, especially at higher thread counts, because the overhead of creating and scheduling tasks becomes more visible when the



(a) Uniform workload



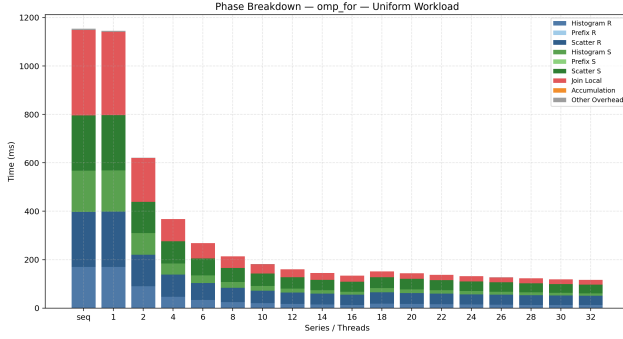
(b) Skewed workload

Figure 2: Weak scaling efficiency of the OpenMP loop-based and task-based implementations under uniform and skewed workloads.

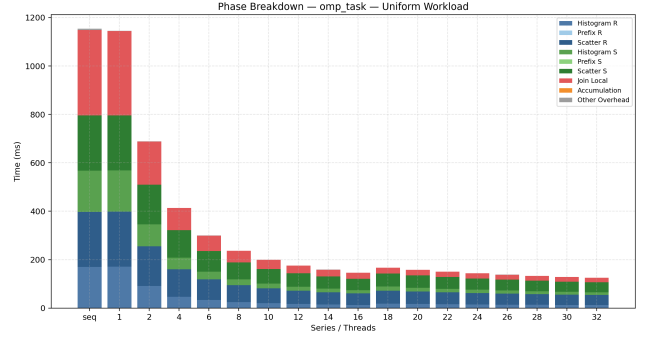
workload is already fairly balanced.

On the skewed workload, the efficiency drop is stronger for both implementations. In this case, increasing the number of threads is less effective because a large fraction of the work is concentrated on a small number of hot partitions. The loop-based version still performs better than the task-based one, confirming that dynamic scheduling over partitions is sufficient to mitigate part of the imbalance in this setting.

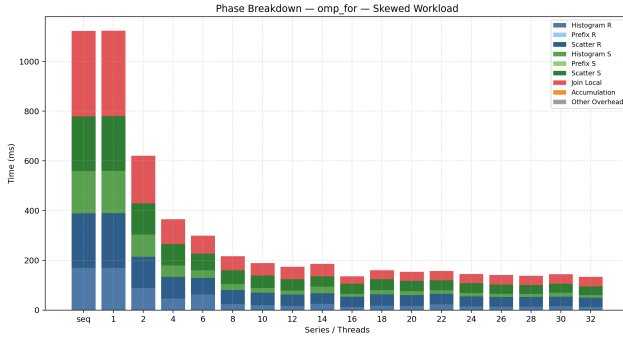
2.3 Phase Breakdown



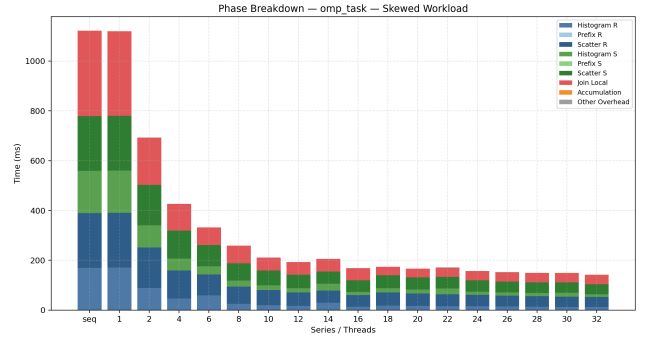
(a) loop-based implementation , uniform workload



(b) task-based implementation , uniform workload



(c) loop-based implementation , skewed workload



(d) task-based implementation , skewed workload

Figure 3: Execution time breakdown by phase for the OpenMP loop-based and task-based implementations under uniform and skewed workloads.

Figure 3 extends the phase breakdown analysis already performed in Module 2 to the two OpenMP implementations and to both workload types. The same general behavior is confirmed: histogram computation, scatter, and local join dominate the execution time, while prefix sum and final accumulation remain marginal.

The scatter phase scales less regularly than histogram and join, since it is more memory-bound and involves irregular writes. This was already visible in the C++ threads implementation and remains one of the main limits also in the OpenMP versions.

The comparison between uniform and skewed workloads shows that skewness mainly affects the local join phase. When many records are mapped to a small number of hot partitions, the join work becomes less evenly distributed. This reduces the available parallelism and explains part of the lower scalability observed in the skewed experiments.



Figure 4: Execution time of the OpenMP implementations as the percentage of records assigned to hot partitions increases, with $T = 8$ threads.

2.4 Impact of Workload Skewness

Figure 4 isolates the effect of workload skewness by fixing the number of threads to $T = 8$. In this experiment, the plot reports the absolute execution time of the two OpenMP implementations as the percentage of records assigned to hot partitions increases.

The loop-based implementation remains faster than the task-based one across the tested skew levels. This suggests that the dynamic scheduling used in the partition-wise join phase is already sufficient to mitigate part of the load imbalance introduced by the skewed workload. The task-based implementation does not provide a clear advantage in this setting, because the additional task management overhead is not compensated by a sufficient improvement in load balancing.

As the skew percentage increases, a larger fraction of the records is mapped to a small subset of hot partitions. Consequently, the local join work becomes less evenly distributed across partitions, reducing the amount of parallelism that can be effectively exploited. The trend is not necessarily monotonic because changing the skew level also changes the actual distribution of keys and matches inside the partitions. For this reason, the skewness experiment should be interpreted together with the phase breakdown, which shows that the local join phase is one of the main components affected by load imbalance.

3 Discussion and Comparison with Module 2

The OpenMP implementations preserve the same algorithmic structure used in Module 2, but replace the explicit C++ thread pool with OpenMP constructs. The results show that the loop-based OpenMP version is the most effective strategy among the two OpenMP implementations.

On uniform workloads, loop-based implementation behaves similarly to the C++ threads implementation from Module 2, reaching approximately the same maximum speedup at 32 threads. This suggests that the parallelism exposed by the algorithm can be expressed efficiently using OpenMP loop-level parallelism.

The task-based version is less competitive in the tested configurations. Its additional scheduling flexibility does not compensate for the task management overhead, especially when the workload is already sufficiently balanced. Under skewness, both OpenMP versions are affected by load imbalance, but loop-based implementation remains more robust.

Overall, the main limitations are the same observed in Module 2: memory-bound scatter operations and load imbalance in the local join phase. Prefix sum and final accumulation remain marginal and do not significantly affect the overall scalability.